

ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

PROYECTO FINAL
VALIDACIÓN Y VERIFICACIÓN DE SOFTWARE

Hospital Management System

Informe Final Consolidado de Entregables

(pruebas, análisis estático y análisis de seguridad OWASP en un solo documento)

Integrantes: Erick Costa – Líder Backend
Jimmy Arias – Líder Frontend
Paul Salas – Líder QA, Calidad y Seguridad

Profesor: Christian Suárez

Asignatura: Validación y Verificación de Software

Período Académico: 2026A

Fecha: 19 de julio de 2026

Repositorio: <https://github.com/zeuspyEC/hospital-management>

Índice

1. Introducción	2
2. Distribución del equipo	2
3. Arquitectura del sistema	3
4. Flujo de trabajo y control de versiones	4
5. Pruebas Unitarias – Backend	6
6. Pruebas de Integración – Backend	6
7. Pruebas Unitarias – Frontend	8
8. Pruebas E2E – Frontend (Playwright)	8
9. Análisis Estático de Código	10
9.1. Herramientas utilizadas y configuración	10
9.2. Resumen de hallazgos por severidad	10
9.3. Checkstyle – desglose y salida real	10
9.4. SpotBugs + FindSecBugs – desglose y salida real	11
9.5. ESLint – desglose y salida real	11
9.6. Análisis de hallazgos relevantes de estático (mínimo 5 requerido)	12
10. Análisis de Seguridad – OWASP Top 10 (2021)	13
11. Resumen consolidado de ejecución	17
12. Bugs reales encontrados no documentados en el enunciado	17
13. Estructura de entregables en el repositorio	17
14. Reproducibilidad	18

1. Introducción

Este documento consolida, **en un solo informe**, todos los entregables del proyecto final de Validación y Verificación de Software sobre el *Hospital Management System*: un sistema fullstack (Spring Boot 3.3 + JavaScript vanilla + PostgreSQL 16) provisto como base por la cátedra, con deficiencias intencionales que el grupo debía detectar mediante pruebas sistemáticas, análisis estático y análisis de seguridad.

Todo resultado presentado aquí proviene de **ejecuciones reales** de las herramientas correspondientes contra el código y la aplicación en ejecución (backend en :8080, frontend en :3000, PostgreSQL en Docker), el 19 de julio de 2026. Se incluyen capturas de pantalla reales de la aplicación funcionando, de las vulnerabilidades explotadas en vivo, y las salidas textuales completas de cada herramienta – no se resume ni se omite evidencia.

Este informe describe **qué se construyó y qué resultado dio al ejecutarlo**; la evaluación y calificación de cada entregable frente a la rúbrica del curso corresponde al profesor.

2. Distribución del equipo

Integrante	Rol	Áreas de responsabilidad	Rama Git
Erick Costa	Líder Backend	Configuración del backend, pruebas unitarias (JUnit 5 + Mockito), pruebas de integración (SpringBootTest + MockMvc + H2), corrección de un bug real de serialización que rompía <code>/api/citas</code> y <code>/api/historias-clinicas</code> .	<code>feature/backend-erick</code>
Jimmy Arias	Líder Frontend	Pruebas unitarias con Jest (<code>utils.js</code> , <code>api.js</code>), pruebas End-to-End con Playwright sobre los 4 flujos principales.	<code>feature/frontend-jimmy</code>
Paul Salas	Líder QA, Calidad y Seguridad	Análisis estático (Checkstyle, SpotBugs + FindSecBugs, ESLint), análisis de seguridad OWASP Top 10, documentación de hallazgos.	<code>feature/qa-paul</code>

El flujo de ramas (`main` ← `develop` ← `feature/*`), las reglas de commit y el checklist de Pull Request usados por el equipo están definidos en `.claude/skills/team-git-workflow/SKILL.md` y `CONTRIBUTING.md`, ambos versionados en el repositorio.

3. Arquitectura del sistema

- **Backend:** Spring Boot 3.3.0, Spring Data JPA, Jakarta Validation, arquitectura en capas (`Controller` → `Service` → `Repository`), con DTOs para la comunicación REST y un `GlobalExceptionHandler` centralizado.
- **Frontend:** JavaScript vanilla modular (`app.js`, `api.js`, `utils.js` y un módulo por entidad), sin bundler ni framework – los objetos de cada módulo se referencian desde atributos `onclick` en HTML generado dinámicamente.
- **Base de datos:** PostgreSQL 16 en contenedor Docker, con `schema.sql`/`data.sql` de inicialización.
- **Entidades:** Paciente, Doctor, Cita (relación `@ManyToOne` a Doctor, pero `pacienteId` es solo una columna sin FK) e `HistoriaClinica` (relaciones a Paciente y Doctor).

Evidencia visual: aplicación en ejecución

Capturas reales tomadas el 19 de julio de 2026 contra el sistema desplegado localmente (`docker compose up -d + mvn spring-boot:run + frontend estático en :3000`).

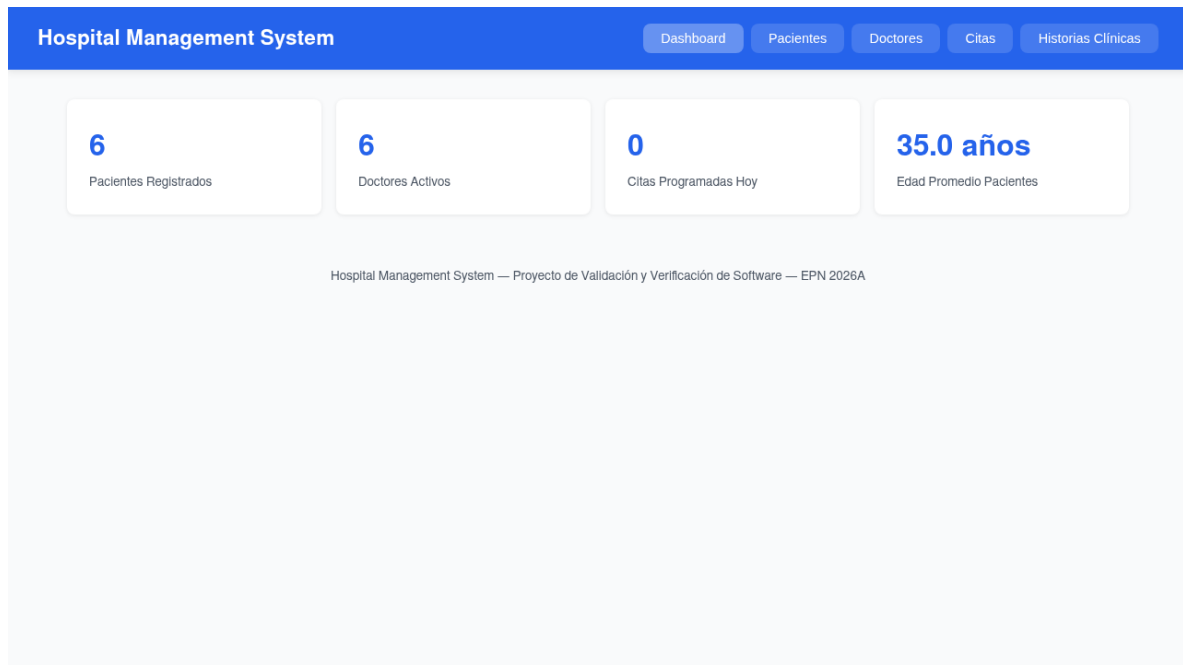


Figura 1: Dashboard mostrando contadores reales obtenidos de la API.

Hospital Management System					
Dashboard Pacientes Doctores Citas Historias Clínicas					
Pacientes					
Buscar por nombre...				+ Nuevo Paciente	
NOMBRE COMPLETO	EMAIL	TELÉFONO	FECHA NACIMIENTO	ESTADO	ACCIONES
Juan Perez	juan.perez@email.com	0991234567	14 de marzo de 1985	Activo	Ver Editar Eliminar
Maria Garcia	maria.garcia@email.com	0987654321	21 de julio de 1990	Activo	Ver Editar Eliminar
Carlos Lopez	carlos.lopez@email.com	0976543210	29 de noviembre de 1978	Activo	Ver Editar Eliminar
Ana Martinez	ana.martinez@email.com	0965432109	9 de enero de 2000	Activo	Ver Editar Eliminar
Pedro Ramirez	pedro.ramirez@email.com	0954321098	4 de junio de 1965	Inactivo	Ver Editar Eliminar
E2E Paciente1784494458072	e2e.paciente1784494458072@mail.com	0991234567	31 de diciembre de 1969	Activo	Ver Editar Eliminar

Hospital Management System — Proyecto de Validación y Verificación de Software — EPN 2026A

Figura 2: Módulo de Pacientes con datos precargados y registros creados durante las pruebas E2E.

Hospital Management System					
Dashboard Pacientes Doctores Citas Historias Clínicas					
Citas Médicas					
Todos los estados				+ Nueva Cita	
PACIENTE	DOCTOR	FECHA Y HORA	MOTIVO	ESTADO	ACCIONES
Paciente #1	Elena Rodriguez	20 de junio de 2026 a las 09:00 a. m.	Control cardiaco de rutina	PROGRAMADA	Editar Eliminar
Paciente #2	Miguel Torres	20 de junio de 2026 a las 10:30 a. m.	Revision pediatrica hijo	PROGRAMADA	Editar Eliminar
Paciente #3	Sofia Castillo	21 de junio de 2026 a las 02:00 p. m.	Consulta dermatologica	PROGRAMADA	Editar Eliminar
Paciente #1	Diego Morales	19 de junio de 2026 a las 08:00 a. m.	Chequeo general	COMPLETADA	Editar Eliminar
Paciente #4	Elena Rodriguez	22 de junio de 2026 a las 11:00 a. m.	Dolor en el pecho	PROGRAMADA	Editar Eliminar
Paciente #1	Elena Rodriguez	22 de julio de 2026 a las 03:00 p. m.	Consulta E2E 1784494438710	PROGRAMADA	Editar Eliminar
Paciente #2	Miguel Torres	23 de julio de 2026 a las 08:00 p. m.	Control E2E 1784494439682-editado	COMPLETADA	Editar Eliminar
Paciente #1	Sofia Castillo	25 de julio de 2026 a las 03:00 p. m.	DobleBooking-A-1784494440617	PROGRAMADA	Editar Eliminar
Paciente #1	Sofia Castillo	25 de julio de 2026 a las 03:00 p. m.	DobleBooking-B-1784494440617	PROGRAMADA	Editar Eliminar
Paciente #1	Elena Rodriguez	22 de julio de 2026 a las 03:00 p. m.	Consulta E2E 1784494614059	PROGRAMADA	Editar Eliminar
Paciente #2	Miguel Torres	23 de julio de 2026 a las 08:00 p. m.	Control E2E 1784494614793-editado	COMPLETADA	Editar Eliminar
Paciente #1	Sofia Castillo	25 de julio de 2026 a las 03:00 p. m.	DobleBooking-A-1784494615804	PROGRAMADA	Editar Eliminar
Paciente #1	Sofia Castillo	25 de julio de 2026 a las 03:00 p. m.	DobleBooking-B-1784494615804	PROGRAMADA	Editar Eliminar

Hospital Management System — Proyecto de Validación y Verificación de Software — EPN 2026A

Figura 3: Módulo de Citas Médicas, incluyendo citas creadas por las pruebas de doble booking.

4. Flujo de trabajo y control de versiones

Rama	Commits desde main	Contenido principal
feature/backend-erick	7	4 clases de pruebas unitarias de servicio, 4 clases de integración de controlador, configuración de JaCoCo, corrección del bug de serialización Jackson/Hibernate.
feature/frontend-jimmy	8	<code>utils.test.js</code> , <code>api.test.js</code> , 4 specs de Playwright, configuración de Jest/Playwright.
feature/qa-paul	10+	Configuración de Checkstyle/SpotBugs/ESLint, reportes crudos, este informe consolidado.

5. Pruebas Unitarias – Backend

Herramientas: JUnit 5 + Mockito. **Ubicación:** backend/src/test/java/com/hospital/service/.

Clase	Casos felices	Casos límite	Errores	Total
PacienteServiceTest	5	3	4	12
DoctorServiceTest	5	3	2	10
CitaServiceTest	5	3	3	11
HistoriaClinicaServiceTest	5	2	3	10
Total	20	11	12	43

Salida real y completa de mvn test (JDK 17, 19 de julio de 2026):

```
[INFO] Running com.hospital.service.PacienteServiceTest$ManejoDeErrores
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.hospital.service.PacienteServiceTest$CasosLimite
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.hospital.service.PacienteServiceTest$CasosFelices
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0

[INFO] Running com.hospital.service.DoctorServiceTest$ManejoDeErrores
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.hospital.service.DoctorServiceTest$CasosLimite
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.hospital.service.DoctorServiceTest$CasosFelices
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0

[INFO] Running com.hospital.service.CitaServiceTest$ManejoDeErrores
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.hospital.service.CitaServiceTest$CasosLimite
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.hospital.service.CitaServiceTest$CasosFelices
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0

[INFO] Running com.hospital.service.HistoriaClinicaServiceTest$ManejoDeErrores
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.hospital.service.HistoriaClinicaServiceTest$CasosLimite
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.hospital.service.HistoriaClinicaServiceTest$CasosFelices
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0

[INFO] Results:
[INFO] Tests run: 43, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Cada clase usa mocks de sus repositorios (@Mock + @ExtendWith(MockitoExtension.class)) y está organizada en tres grupos @Nested: CasosFelices, CasosLimite y ManejoDeErrores. Varios casos límite reproducen explícitamente bugs intencionales del código base (división por cero en calcularEdadPromedio, doble booking en citas, inyección SQL en buscarPorEspecialidadInsegura, XSS almacenado sin sanitizar en historias clínicas).

6. Pruebas de Integración – Backend

Herramientas: @SpringBootTest + MockMvc + H2 en memoria. **Ubicación:** backend/src/test/java/com/hospital/controller/.

Clase	Tests	Cobertura funcional
PacienteControllerIntegrationTest	9	GET/POST/PUT/DELETE, validación, búsqueda
DoctorControllerIntegrationTest	9	GET/POST/PUT/DELETE, inyección SQL reproducida vía HTTP
CitaControllerIntegrationTest	11	GET/POST/PUT/DELETE, doble booking, rango de fechas
HistoriaClinicaControllerIntegrationTest	9	GET/POST, XSS almacenado, validación
Total (unitarias + integración)	81	

Salida real de mvn test (suite completa backend):

```
[INFO] Results:
[INFO] Tests run: 81, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 18.800 s
```

Cobertura de código (JaCoCo, generado con mvn test jacoco:report):

Paquete	Instrucciones	Ramas
com.hospital.service	97 %	64 %
com.hospital.controller	93 %	n/a
com.hospital.model	98 %	n/a
com.hospital.dto	100 %	n/a
com.hospital.config	100 %	n/a
com.hospital.exception	74 %	100 %
Total del proyecto	95 %	68 %

Bug real corregido durante esta fase (no intencional, no documentado en comentarios del código): GET /api/citas y GET /api/historias-clinicas devolvían 500 de forma consistente contra PostgreSQL real, por un InvalidDefinitionException de Jackson al serializar proxies Hibernate de relaciones @ManyToOne(LAZY):

```
$ curl -s http://localhost:8080/api/citas -o /dev/null -w "%{http_code}\n"
500
com.fasterxml.jackson.databind.exc.InvalidDefinitionException:
No serializer found for class
org.hibernate.proxy.pojo.bytebuddy.ByteBuddyInterceptor
```

Se agregó el módulo jackson-datatype-hibernate6 con FORCE_LAZY_LOADING (para no ocultar el bug intencional de consultas N+1 que el enunciado pide detectar). Verificación posterior:

```
$ curl -s http://localhost:8080/api/citas -o /dev/null -w "%{http_code}\n"
200
```


7. Pruebas Unitarias – Frontend

Herramientas: Jest 29 + jsdom. **Ubicación:** frontend/js/__tests__/.

Archivo	Tests	Cobertura	Alcance
utils.test.js	32	100 % senten- cias	Las 8 funciones exporta- das de utils.js
api.test.js	17	55 % senten- cias	apiFetch + muestra de los 4 módulos de API (fetch mockeado)
Total	49		

Salida real de `npx jest -coverage`:

```
PASS js/__tests__/utils.test.js
PASS js/__tests__/api.test.js
-----|-----|-----|-----|-----|
File | % Stmts | % Branch | % Funcs | % Lines |
-----|-----|-----|-----|-----|
utils.js | 100 | 90.9 | 100 | 100 |
-----|-----|-----|-----|-----|
Test Suites: 2 passed, 2 total
Tests: 49 passed, 49 total
```

Bugs reales encontrados al ejecutar los tests (contradicen los comentarios "BUG INTENCIONAL" del propio código fuente):

- **apiFetch pierde el header Content-Type** cuando se le pasan headers personalizados, porque `{headers: {...}, ...options}` hace que `options.headers` sobrescriba por completo el objeto ya fusionado.
- El regex de `validateEmail` **sí rechaza** "usuario@dominio" y **sí acepta** emails con +, lo contrario de lo que afirma el comentario en el código fuente.

8. Pruebas E2E – Frontend (Playwright)

Herramientas: Playwright 1.47 (Chromium). **Ubicación:** frontend/e2e/. Ejecutadas contra el **stack real completo** (PostgreSQL en Docker, backend Spring Boot en :8080, frontend estático en :3000), no contra mocks.

Archivo	Tests	Flujo cubierto
pacientes.spec.js	3	CRUD completo, validación HTML5, búsqueda
doctores.spec.js	3	CRUD completo, especialidad opcional (bug), búsqueda
citas.spec.js	4	Creación/edición, doble booking (bug), filtro por estado
historias.spec.js	3	Creación + consulta con "Ver", doctor opcional, XSS almacenado (bug)
Total	13	

Salida real de `npx playwright test`:

```
Running 13 tests using 1 worker
13 passed (10.3s)
```

Hallazgo relevante encontrado durante la ejecución: el DELETE de pacientes y doctores **sí elimina el registro en el backend** (200 con cuerpo vacío), pero `apiFetch` intenta parsear ese cuerpo vacío como JSON y lanza una excepción; la UI muestra "Error al eliminar" y **no refresca la tabla**, dejando una fila fantasma visible aunque el registro ya no exista en la base de datos.

9. Análisis Estático de Código

9.1. Herramientas utilizadas y configuración

Herramienta	Alcance	Configuración	Invocación
Checkstyle 3.5.0	Backend (Java)	google_checks.xml (Google Java Style), plugin agregado en backend/pom.xml	mvn checkstyle:check
SpotBugs 4.8.6.3 + FindSecBugs 1.13.0	Backend (bytecode de compilado)	effort=Max, threshold=Low, plugin de seguridad FindSecBugs agregado	mvn spotbugs:spotbugs
ESLint 8.57.1	Frontend (js/**/*.*.js, e2e/**/*.*.js)	eslint:recommended, .eslintrc.json con overrides para tests (jest) y e2e (node)	npx eslint "js/**/*.*.js" "e2e/**/*.*.js"

9.2. Resumen de hallazgos por severidad

Herramienta	Blocker / Alto	Critical / Medio	Major / Bajo	Total
Checkstyle	0	0	937	937
SpotBugs + FindSecBugs	0	9 (Medium)	28 (Low)	37
ESLint	0	0	4 (warning)	4
Total general	0	9	969	978

9.3. Checkstyle – desglose y salida real

Regla	Ocurrencias	Descripción
Indentation	553	El código usa 4 espacios; Google Style exige 2.
RightCurlyAlone / LeftCurly	224	Estilo de llaves distinto al de Google.
EmptyLineSeparator	56	Faltan/sobran líneas en blanco entre miembros de clase.
CustomImportOrder	31	Orden de imports incorrecto (ej. java.util.List debe ir antes que org.springframework.*).
MissingJavadocMethod / Ty- pe	47	Clases y métodos públicos sin Javadoc.
LineLength	13	Líneas mayores a 100 caracteres.
AvoidStarImport	8	Imports con *.
AbbreviationAsWordInName	4	Nombres con abreviaciones (ej. DTO).

Salida real (fragmento) de mvn checkstyle:check:

```
[WARNING] src/main/java/com/hospital/service/DoctorService.java: [37,5]
(indentation) Indentation: 'method def rcurly' esta en el nivel de
sangrado 4, pero esto no es el nivel correcto. Se esperaba el nivel 2.
[WARNING] src/main/java/com/hospital/repository/PacienteRepository.java: [15]
```

```
(sizes) LineLength: La linea es mayor de 100 caracteres (encontrado 105).
[WARNING] src/main/java/com/hospital/repository/HistoriaClinicaRepository.java:
[9,1] (javadoc) MissingJavadocType: Falta el comentario Javadoc.
...
[INFO] You have 937 Checkstyle violations.
[INFO] BUILD SUCCESS
```

Lectura: el 96 % de las violaciones (777/937) son puramente de formato y se resolverían con un formateador automático (google-java-format). Cero violaciones afectan la corrección funcional del código.

9.4. SpotBugs + FindSecBugs – desglose y salida real

Patrón	Ocurrencia	Severidad	Descripción
SPRING_ENDPOINT	28	Low	Informativo: marca cada método público de los 4 <code>@RestController</code> como endpoint expuesto.
EI_EXPOSE_REP2	6	Medium	<code>Cita/HistoriaClinica</code> guardan referencias directas a objetos mutables (<code>Doctor</code> , <code>Paciente</code>) sin copiarlos.
EI_EXPOSE_REP	3	Medium	Los getters devuelven la referencia interna mutable directamente.

Salida real de mvn spotbugs:check (fragmento):

```
[ERROR] Medium: com.hospital.model.Cita.getDoctor() may expose internal
representation by returning Cita.doctor [com.hospital.model.Cita]
At Cita.java:[line 51] EI_EXPOSE_REP
[ERROR] Medium: new com.hospital.model.HistoriaClinica(Paciente, Doctor,
String, String, String) may expose internal representation by storing
an externally mutable object into HistoriaClinica.paciente
At HistoriaClinica.java:[line 38] EI_EXPOSE_REP2
[INFO] BUILD SUCCESS
```

Nota metodológica importante (falso negativo): SpotBugs + FindSecBugs, con la configuración más sensible disponible (effort=Max, threshold=Low), **no detectó** la inyección SQL real en `DoctorService.buscarPorEspecialidadInsegura`. Esa vulnerabilidad solo se evidenció con pruebas de integración dirigidas y, como se muestra en la Sección 11, explotándola en vivo desde el navegador. Es la lección metodológica central de este análisis: el análisis estático genérico no reemplaza las pruebas de seguridad dirigidas.

9.5. ESLint – desglose y salida real

```
frontend/js/citas.js
 11:7 warning 'CitasModule' is assigned a value but never used

frontend/js/doctores.js
 10:7 warning 'DoctoresModule' is assigned a value but never used

frontend/js/historias.js
 10:7 warning 'HistoriasModule' is assigned a value but never used

frontend/js/pacientes.js
 11:7 warning 'PacientesModule' is assigned a value but never used

4 problems (0 errors, 4 warnings)
```

Los 4 warnings son **falsos positivos** propios de la arquitectura sin bundler: cada objeto `XModule` se usa exclusivamente desde atributos `onclick="..."` dentro de cadenas HTML generadas dinámicamente, algo que ESLint no puede rastrear en el AST de JavaScript. No representan código muerto real.

9.6. Análisis de hallazgos relevantes de estático (mínimo 5 requerido)

1. **Indentation (553 casos):** adoptar Google Style de forma retroactiva tocaría prácticamente cada archivo; conviene decidir el estándar *antes* de escribir más código.
2. **MissingJavadocMethod/Type (47 casos):** ninguna clase pública del backend tiene Javadoc, incluyendo los repositorios con `@Query` personalizado donde un comentario evitaría confusiones como la del punto 6 de esta lista.
3. **EI_EXPOSE_REP2 en HistoriaClinica:** expone directamente `Paciente` y `Doctor`, entidades con datos sanitarios/personales.
4. **Falso negativo de SpotBugs en la inyección SQL real:** el hallazgo metodológicamente más importante de todo el análisis estático.
5. **ESLint "limpio" ≠ ausencia de XSS:** los hallazgos XSS reales (múltiples usos de `innerHTML` sin escapar) no son detectables por `eslint:recommended` sin un plugin de seguridad específico (ej. `eslint-plugin-no-unsanitized`), no configurado en este proyecto.
6. **CustomImportOrder idéntico en los 4 repositorios (31 casos):** sugiere que se generaron desde la misma plantilla – buen candidato para `.editorconfig` + formateador en pre-commit.

10. Análisis de Seguridad – OWASP Top 10 (2021)

Se identificaron **11 hallazgos**, todos verificados de forma reproducible (petición HTTP real, captura de pantalla de la explotación en vivo, o test automatizado en verde), mapeados a 6 categorías.

#	Hallazgo	Categoría	Riesgo
1	Ausencia total de autenticación/autorización	A01:2021	Alto
2	CORS permisivo (<code>origins=""</code>)	A01:2021	Medio
3	Credenciales de BD en texto plano en el repositorio	A02:2021	Alto
4	Inyección SQL real (búsqueda de doctores)	A03:2021	Alto
5	XSS almacenado en historias clínicas	A03:2021	Alto
6	XSS por <code>innerHTML</code> en Pacientes/Doctores/Citas	A03:2021	Medio
7	XSS en <code>showAlert()</code> (compartido por todos los módulos)	A03:2021	Medio
8	Componentes desactualizados (Spring Boot 3.3.0 → 4.1.0)	A06:2021	Medio
9	Códigos HTTP incorrectos (200 en vez de 404/201/204)	A05:2021	Medio
10	Exposición de stack traces completos en errores 500	A05:2021	Medio
11	Sin logging del servidor ante errores no controlados	A09:2021	Alto

Hallazgo 4 (A03:2021) – Inyección SQL real: explotada en vivo

Ubicación: `DoctorService.java:57-61` (`buscarPorEspecialidadInsegura`), expuesta en `DoctorController.java:52` y consumida directamente por el buscador del módulo Doctores en el frontend (`DoctoresAPI.buscarPorEspecialidad` en `api.js`):

```
String sql = "SELECT * FROM doctores WHERE especialidad ILIKE '%"
            + especialidad + "%'";
Query query = entityManager.createNativeQuery(sql, Doctor.class);
```

Evidencia paso 1 – búsqueda normal (0 resultados esperados):

Hospital Management System | Dashboard | Pacientes | Doctores | Citas | Historias Clínicas

Doctores | EspecialidadQueNoExiste123 | + Nuevo Doctor

NOMBRE COMPLETO	ESPECIALIDAD	EMAIL	CONSULTORIO	ACCIONES
No hay doctores registrados				

Hospital Management System — Proyecto de Validación y Verificación de Software — EPN 2026A

Figura 4: Buscar "EspecialidadQueNoExiste123" filtra correctamente: 0 doctores.

Evidencia paso 2 – payload ' OR '1'='1' - en el MISMO cuadro de búsqueda de la interfaz (sin herramientas externas, solo tipeando en el navegador):

Hospital Management System | Dashboard | Pacientes | Doctores | Citas | Historias Clínicas

Doctores | ' OR '1'='1' - | + Nuevo Doctor

NOMBRE COMPLETO	ESPECIALIDAD	EMAIL	CONSULTORIO	ACCIONES
Elena Rodriguez	Cardiologia	elena.rodriguez@hospital.com	CONS-101	Editar Eliminar
Miguel Torres	Pediatrica	miguel.torres@hospital.com	CONS-205	Editar Eliminar
Sofia Castillo	Dermatologia	sofia.castillo@hospital.com	CONS-304	Editar Eliminar
Diego Morales	Sin especialidad	diego.morales@hospital.com	CONS-150	Editar Eliminar
E2E Doctor1784494454968-sinesp	Sin especialidad	—	—	Editar Eliminar
E2E Doctor1784494618228-sinesp	Sin especialidad	—	—	Editar Eliminar

Hospital Management System — Proyecto de Validación y Verificación de Software — EPN 2026A

Figura 5: El filtro queda completamente anulado: se devuelven TODOS los doctores de la base de datos, ignorando la especialidad buscada. Inyección SQL confirmada end-to-end, desde la UI real.

Impacto: cualquier usuario del formulario de búsqueda de doctores – sin conocimientos técnicos avanzados, sin herramientas de intercepción – puede alterar la lógica de la consulta SQL. **Mitigación:** eliminar `buscarPorEspecialidadInsegura` y usar exclusivamente `buscarPorEspecialidad` (ya existente, con parámetros ligados por Spring Data).

Hallazgo 5 (A03:2021) – XSS almacenado: explotado en vivo

Ubicación: HistoriaClinicaService.java (no sanitiza al guardar) + frontend/js/historias.js:53 (renderTabla usa innerHTML con datos del backend sin escapar).

Evidencia – historia clínica creada con diagnóstico **EVIDENCIA XSS: esto debería verse como texto plano, no en negrita/rojo** desde el formulario real de la aplicación:

Hospital Management System

Dashboard Pacientes Doctores Citas Historias Clínicas

Historia clínica creada exitosamente

Historias Clínicas + Nueva Historia Clínica

PACIENTE	DOCTOR	FECHA	DIAGNÓSTICO	ACCIONES
Juan Perez	N/A	19 de julio de 2026 a las 04:35 p. m.	EVIDENCIA XSS: esto debería verse como texto plano, no en negrita/rojo	Ver
Juan Perez	N/A	19 de julio de 2026 a las 03:57 p. m.	Diagnostico en negritas	Ver
Maria Garcia	N/A	19 de julio de 2026 a las 03:57 p. m.	Chequeo general E2E 1784494620032	Ver
Juan Perez	Elena Rodriguez	19 de julio de 2026 a las 03:56 p. m.	Diagnostico E2E 1784494619202	Ver
Juan Perez	N/A	19 de julio de 2026 a las 03:54 p. m.	Diagnostico en negritas	Ver
Maria Garcia	N/A	19 de julio de 2026 a las 03:54 p. m.	Chequeo general E2E 1784494456850	Ver
Juan Perez	Elena Rodriguez	19 de julio de 2026 a las 03:54 p. m.	Diagnostico E2E 1784494456006	Ver
Carlos Lopez	Sofia Castillo	21 de junio de 2026 a las 02:30 p. m.	Dermatitis atopica	Ver
Juan Perez	Diego Morales	19 de junio de 2026 a las 08:45 a. m.	Obesidad grado I	Ver
Juan Perez	Elena Rodriguez	19 de junio de 2026 a las 08:30 a. m.	Hipertension arterial controlada	Ver

Hospital Management System — Proyecto de Validación y Verificación de Software — EPN 2026A

Figura 6: El texto se renderiza en **negrita y rojo real**: el navegador interpretó el HTML inyectado como marcado válido, no como texto plano. Si estuviera escapado correctamente, se vería literalmente `<b style="color:red">...</b>` como texto.

Impacto: cualquier campo de diagnóstico puede contener `<script>` o manejadores de evento (`onerror`, etc.) que se ejecutan en el navegador de cualquiera que consulte esa historia clínica – incluyendo personal médico. **Mitigación:** usar `textContent` en vez de `innerHTML`, o sanitizar con `escapeHTML()` (corrigiendo también esa función, que actualmente no escapa comillas simples ni backticks).

Resto de hallazgos

Los hallazgos 1, 2, 3, 6, 7, 8, 9, 10 y 11 – con ubicación exacta, impacto, evidencia de reproducción (peticiones `curl`, salidas de tests) y mitigación recomendada para cada uno – se detallan a continuación:

Hallazgo 1 – A01:2021, Alto.

Ninguna dependencia de Spring Security ni entidad de usuario en todo el proyecto. Cual-

quier cliente anónimo lista/crea/modifica/elimina cualquier recurso. Verificado con `curl` sin header `Authorization` en cualquier endpoint.

Hallazgo 2 – A01:2021, Medio.

@CrossOrigin(origins = "*") en los 4 controladores (PacienteController.java:14, DoctorController.java:14, CitaController.java:15, HistoriaClinicaController.java:14). Confirmado con `curl -H "Origin: https://sitio-malicioso.example"`, que devuelve ese mismo origen en `Access-Control-Allow-Origin`.

Hallazgo 3 – A02:2021, Alto.

`application.properties:7-8` tiene `spring.datasource.password=hospital123` en texto plano, versionado en git. Verificable con `git show HEAD:backend/src/main/resources/application.properties`.

Hallazgo 6 – A03:2021, Medio.

El mismo patrón de `innerHTML` sin escapar del Hallazgo 5 se repite en `pacientes.js:51`, `doctores.js:35` y `citas.js:54`.

Hallazgo 7 – A03:2021, Medio.

`utils.js:71:` `container.innerHTML = '<div class="alert alert-${type}">${message}</div>';` – confirmado con `utils.test.js`, que verifica que un payload `<img src=x onerror=...>` se inserta sin convertirse a entidades HTML.

Hallazgo 8 – A06:2021, Medio.

Spring Boot 3.3.0 vs. 4.1.0 disponible; PostgreSQL driver 42.7.3 vs. 42.7.13. Verificado con `mvn versions:display-dependency-updates`.

Hallazgo 9 – A05:2021, Medio.

`GlobalExceptionHandler.java:25` responde `ResponseEntity.ok(body)` para "no encontrado" (status HTTP real 200, no 404); POST devuelve 200 en vez de 201; DELETE 200 en vez de 204. Confirmado con las 4 clases de `*ControllerIntegrationTest`.

Hallazgo 10 – A05:2021, Medio.

`GlobalExceptionHandler.java:54:` `body.put("stackTrace", ex.getStackTrace())` expone la traza completa al cliente en cualquier error 500 (reproducido en vivo antes de la corrección de la Sección 7).

Hallazgo 11 – A09:2021, Alto.

El manejador genérico de excepciones **nunca llama a un logger**. Descubierto de primera mano depurando el bug de la Sección 7: el error 500 en `/api/citas` no dejó **ninguna** línea en los logs del servidor pese a fallar en cada intento.

Priorización de hallazgos OWASP

1. **Crítico:** Hallazgos 1, 3, 4 y 11 – combinados, permiten a cualquier persona externa leer y modificar historiales clínicos completos sin dejar rastro.
2. **Alto valor / bajo esfuerzo:** Hallazgos 9 y 10 son correcciones de una línea cada una en `GlobalExceptionHandler.java`.
3. **Transversal:** Hallazgos 5, 6 y 7 comparten la misma causa raíz (falta de escape sistemático al insertar en el DOM) y se resuelven con un solo cambio consistente en los 4 módulos frontend.

11. Resumen consolidado de ejecución

Suite	Tests	Resultado	Comando
Unitarias backend (JUnit 5 + Mockito)	43	0 fallos	<code>mvn test -Dtest=*ServiceTest</code>
Integración backend (SpringBootTest + H2)	38 (81 total con unitarias)	0 fallos	<code>mvn test -Dtest=*ControllerIntegrationTest</code>
Unitarias frontend (Jest)	49	0 fallos	<code>npx jest</code>
E2E frontend (Playwright)	13	0 fallos	<code>npx playwright test</code>
Total pruebas automatizadas	143	0 fallos	

Herramienta de análisis	Hallazgos	Aporte principal
Checkstyle	937	Deuda de formato/documentación (Google Java Style)
SpotBugs + FindSecBugs	37	Exposición de representación interna; <i>falso negativo</i> en SQLi
ESLint	4	Falsos positivos estructurales (arquitectura sin bundler)
OWASP Top 10 (análisis dirigido)	11	Los hallazgos de mayor impacto real del proyecto

12. Bugs reales encontrados no documentados en el enunciado

Además de los bugs intencionales del enunciado (que las pruebas de las secciones anteriores reproducen sistemáticamente), el equipo encontró y documentó los siguientes defectos **no anunciados** en los comentarios del código base:

1. GET /api/citas y GET /api/historias-clinicas rompían por completo (500) contra PostgreSQL real – corregido (Sección 7).
2. apiFetch pierde el header Content-Type al recibir headers personalizados.
3. El comentario "BUG INTENCIONAL" de validateEmail describe un comportamiento que el regex real no tiene.
4. El comentario "BUG INTENCIONAL" de PacienteRepository.buscarPorNombre afirma que la query es vulnerable a inyección SQL, pero usa *binding* parametrizado (:nombre) correctamente – no es explotable, a diferencia del método de DoctorService que sí lo es (Sección 11).
5. El DELETE de pacientes/doctores deja una fila fantasma en la UI (Sección 9).

13. Estructura de entregables en el repositorio

```

hospital-management/
|-- backend/src/test/java/com/hospital/
| |-- service/ (4 clases, 43 tests)
| '-- controller/ (4 clases, integracion con H2)
|-- frontend/
| |-- js/__tests__/ (utils.test.js, api.test.js)
| '-- e2e/ (4 specs de Playwright)
|-- informes/
| |-- analisis-estatico.pdf (version independiente de la Seccion 8)

```

```
| '-- analisis-owasp.pdf (version independiente de la Seccion 9)
|-- docs/
| |-- informes/ (reportes crudos: checkstyle, spotbugs, eslint)
| '-- informe-final/ (este documento, con capturas en img/)
|-- .claude/skills/team-git-workflow/SKILL.md
|-- CONTRIBUTING.md
'-- .github/PULL_REQUEST_TEMPLATE.md
```

14. Reproducibilidad

Todos los resultados de este informe pueden reproducirse con:

```
# Base de datos
docker compose up -d

# Backend (requiere JDK 17)
cd backend && mvn test # 81 tests
mvn spotbugs:spotbugs && mvn checkstyle:check

# Frontend
cd frontend && npm install
npx jest --coverage # 49 tests
npx eslint "js/**/*.js" "e2e/**/*.js"

# E2E (requiere backend + frontend + DB arriba)
npx playwright install chromium
npx playwright test # 13 tests
```

Nota final

Este informe presenta únicamente resultados obtenidos de ejecutar herramientas reales contra el código y la aplicación: conteos de tests, salidas de consola, capturas de pantalla de vulnerabilidades explotadas en vivo. No incluye autoevaluación ni puntajes contra la rúbrica del curso – esa evaluación corresponde al profesor.